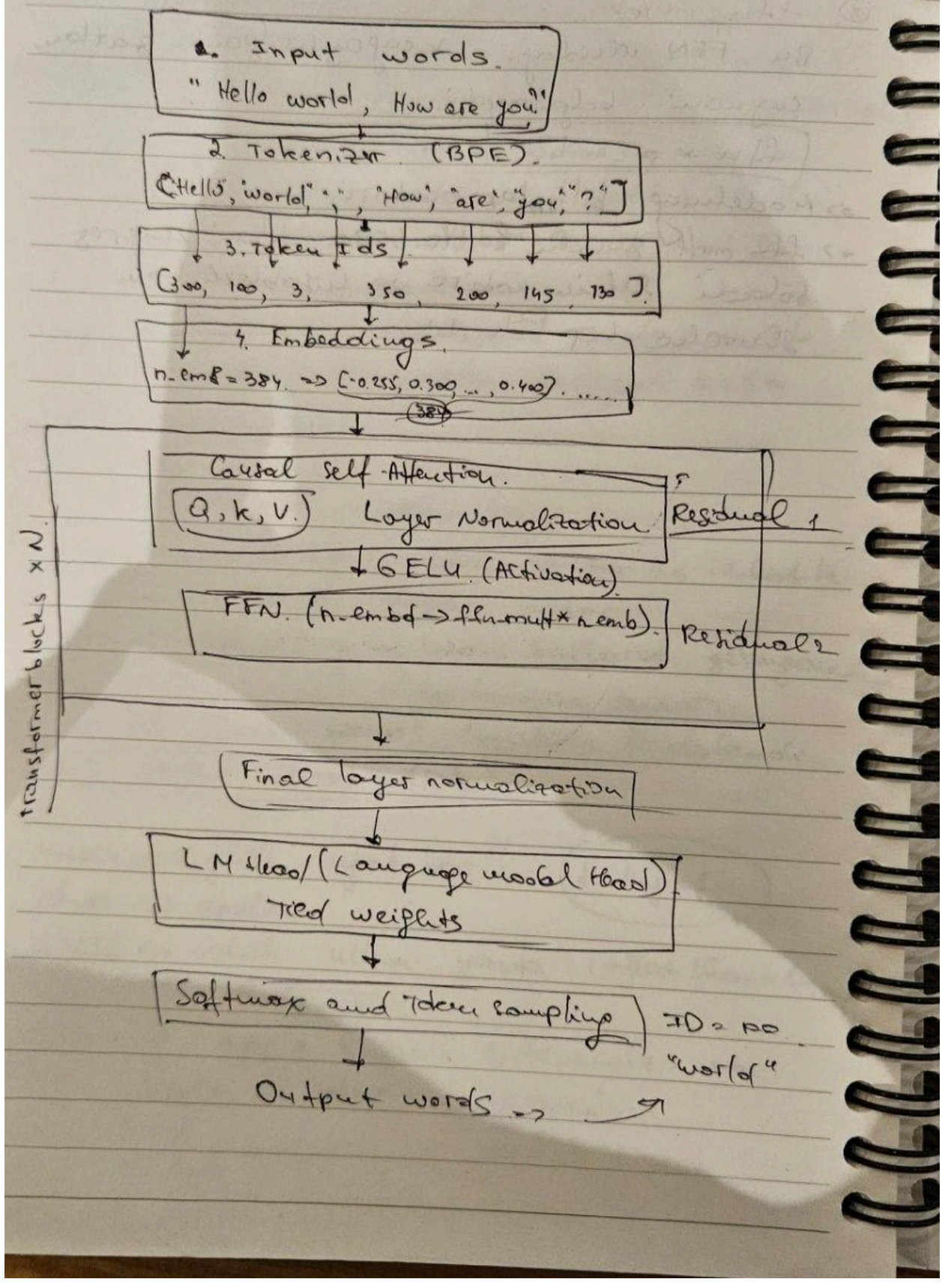
LLM Transformer Model Architecture



Attention

Gapdagi sözlarga bir biri bilan gaplashish imkonini beradi. shu orgali model butun contextni tushunadi.

"Daryo bo'yidagi bank" bank -> river
"Pulni bankka go'ydim" bank -> money

Embeddingda "bank" sozining vectori hamma gapda hamma bir xil. Attentiondan keyin ular har xil bo'ladi. Chunki contextga qarab ~~gay~~ o'z manosini aniqlab oladi shu sababli **Attention => dynamic**

Casual self Attention - self Attention turi. o'zidan oldingi sozga qaray oladi lekin keyingilari bloklanadi.

	Daryo	bo'yidagi	bank.
Daryo	✓	✗	✗
bo'yidagi	✓	✓	✗
bank	✓	✓	✓

=> Trainingda model keyingi sozni bashorat qilishni organadi. Agar u keyingi sozni ko'ra olsa, javobni ko'chirib oladi va hech narsa o'rganmaydi.

Global => Har bir so'z gapdagi barcha so'zlarga qaraydi. Oldingi va keyingi.

Reading files, Images, Code => Global
Generating Text, Chatting, Math reasoning => Casual

Tushunish uchun Global,
yaratish uchun Casual

FFN

Attentionda so'zlar qaysi so'zga qancha qarashi har bir gap uchun yangidan hisoblanadi.
FFN esa har bir tokenga bir xil formulani qo'llaydi training tugagach ularning weights qotib qoladi. va har qanday gapda o'zgarmaydi.

"Daryo bo'yidagi bank"
Attentiondan keyin "bank" vektori bank=river
ekanini bilolmaydi. Endi FFN unga bilim qo'shadi
input => 384 dimension vector.
Bu kengaytiriladi => 4x => 1536
1536ta neyron bor. Har bir neyronni bittta
savolni tekshiradigan detektor deb tasavvur
qilsak bo'ladi.
=> GELU activation function qiymatlarni ega
yoqin qiladi.

Nimaga kengaytirish kerak?

384 -> 384 384 ta saval (neyron)

384 -> 1536 1536 ta saval (neyron)

ko'proq saval bilan ko'proq bilim olinadi.

=> keng joyda ajratish osonroq.

Tor joyda turli patterns bir birige aralashib ketadi keng joyda esa ular alohida alohida joylashadi. GeLU kerakligini "yog'ish", karakterini "o'lim" oladi.

384 -> [1536: keng joyda o'ylash] -> 384 (bilgelikda)

Dropout => Training paytida FFN chiqishidagi sonlarning 10% i tasodifiy 0 ga aylantiriladi.

dropout = 0.1.

=> Model bitta neyronga haddan tashqari tayanib qolmasligi uchun. overfittingni kamaytiradi.

residual => FFN natijasi eski vektorning o'rnini egallamaydi balki unga qo'shiladi.

$x_{new} = x + FFN(LN(x))$.

=> Eski ma'lumot yo'qolmaydi.

=> Dropout trainingda modelni bitta neyronga tayanib qolishidan saqlaydi, Residual esa FFN topgan bilimni eski ma'lumotni o'chirmasdan unga qo'shadi.